Teammate Finder

Jeremy Ellis, Mugi Murugavel, Nitant Rai, Isaac Han

Point of contact email addresses:

JeremyEllis@vt.edu, mugi@vt.edu, nitant3@vt.edu, sehyunhan@vt.edu

CS 4604, Dr. Nizamani

Spring 2026

Table of Contents

1. Introduction to the Teammate Finder....3
1.1 Background....3
1.2 Motivation....3
1.3 Overview of the Database....3
2. User Support....5
2.1 User Roles and Entities....5
2.2 Authentication (Login System)....6
2.3 Security Measures....8
3. Database Connection & Setup....10
3.1 System Connection....10
3.2 Setup Instructions....11
3.3 Database Initialization....11
3.4 Challenges & Lessons Learned....11
4. System Design & Functionality....12
4.1 Schema Design....12
4.2 ER Diagram....13
4.3 Tables & Keys....14
4.3 Example Queries....18
5. Reporting....19
6. GUI / Interface....22
6.1 CRUD Operations....22
Fig. 22: Create Group Modal....22
7. Reflection....25
7.1 Individual Contributions....25
7.2 Team Reflection....25
8. Appendix....26

1. Introduction to the Teammate Finder

1.1 Background

Provide context for your project. Describe the domain, problem area, or real-world scenario your system is based on. Clearly explain what kind of problem your system addresses.

In academic environments, finding the right partners for group assignments can be difficult. This project brings educational technology focusing on the logistics of student collaboration. It is usually difficult for students to identify teammates whose skills complement their own, and this project solves that challenge.

1.2 Motivation

Explain why you built this project and who it is intended for. Describe the goals of your system and how it helps users. Identify the key actions users will perform and why those actions are important.

We built this platform because trying to find teammates for group projects can be frustrating. The system is meant for both teachers and students to use. Teachers can create groups for their specific classes, and then students can go in and make posts saying they are looking for partners. The main goal of our system is to take the guesswork out of building a team by using skill-based matchmaking to sort everyone's posts. The most important thing users do is interact with these ranked posts; the system automatically pushes students with matching skills to the top of the screen, making it easy to find someone who is compatible with what your team needs.

1.3 Overview of the Database

Main entities and data stored:

- Accounts — user profiles storing name, email, hashed password, admin status, and registration timestamp
- Skills — a catalog of technical competencies (e.g. Python, React, PostgreSQL), each categorized as a language, framework/library, database, cloud platform, or tool
- Classes — university courses storing a course name, section number, and the account that created them
- Posts — recruitment listings within a class, storing a title, description, author, and an optional link to a group
- Groups — project teams storing a name, max member cap, the class they belong to, and membership records with a pending/accepted status per member
- Password Reset Tokens — short-lived records storing a secure token tied to an account with an expiration timestamp for account recovery

How users interact with the system:

Users register and log in to the platform. They add skills to their profile to signal their technical background. They browse classes and read posts from other students looking for group members. They can create their own posts to recruit teammates or request to join an existing group. Admins can manage users and platform content.

Key relationships between entities:

- An account can have many skills (many-to-many via account_skills)
- A class has many posts and many groups
- A post belongs to one class and one author, and is optionally linked to a group
- A group belongs to one class and can have many members (many-to-many via account_groups, with a pending flag for join requests)
- A group has a max member cap enforced at the database level

2. User Support

2.1 User Roles and Entities

There are two roles: Admin and Regular User. Both roles share the same accounts entity. Admin is simply a boolean flag (is_admin) on the account, not a separate entity.

Regular User

- Register, log in, and manage their own account (including password reset)
- Add and remove skills from their profile
- Create classes and browse all classes
- Create posts within any class to recruit group members
- Request to join groups, and accept or remove members from groups they own
- Delete their own posts and classes

Admin

- Everything a regular user can do, plus:
- View all user accounts
- Delete any user account, post, or class
- Promote any user to admin

The system intentionally does not separate "teacher" and "student" roles. Any user can create a class or create a post within a class, keeping the platform flexible; a course instructor can post just as easily as a student, and a student can organize a class if needed.

Fig. 1: User Entity Attributes

Attribute	Type	Notes
id	integer	Primary key, auto-generated
email	text	Unique, used for login
password_hash	text	Bcrypt-hashed password
name	text	Display name
is_admin	boolean	Defaults to false
created_at	timestamp	Auto-set on registration

2.2 Authentication (Login System)

Account Creation (Signup)

When a user registers, they provide a name, email, and password. The password is immediately hashed using bcrypt with a cost factor of 10 before being stored. The plaintext password never touches the database. If the email is already taken, a duplicate key error is returned. On success, a JWT is issued and set as an httpOnly cookie, logging the user in immediately.

Login

The user submits their email and password. The backend looks up the account by email and compares the submitted password against the stored bcrypt hash using bcrypt.compare. To prevent timing attacks (where an attacker can tell whether an email exists based on response time), a dummy hash is always compared against, even when no account is found, ensuring login attempts take the same amount of time regardless. On success, a signed JWT is issued with a 7-day expiry and stored in an httpOnly cookie.

Session Persistence

On page refresh, the frontend calls /accounts/login/me. If the JWT cookie is present and valid, the server returns the current user; no re-login required.

Logout

Logout clears the JWT cookie server-side, immediately invalidating the session.

Password Reset

Users can request a password reset by submitting their email. A cryptographically random token is generated and stored in password_reset_tokens with a 1-hour expiry. A reset link containing the token is emailed to the user. When submitted, the token is validated and deleted (one-time use), and the new password is bcrypt-hashed before being saved.

Fig. 2: Security Requirements

Requirement	Implementation
No hardcoded passwords	All secrets (JWT secret, email credentials) are loaded from environment variables, never in source code
Secure password storage	Passwords are hashed with bcrypt (cost factor 10), only the hash is stored
Timing attack prevention	A dummy hash is compared even when no account is found, so response time is constant

Cookie security	JWT is stored in an httpOnly cookie (inaccessible to JavaScript), set to secure and sameSite: none in production
Token expiry	JWTs expire after 7 days; password reset tokens expire after 1 hour and are deleted on use

Fig. 3: creating account

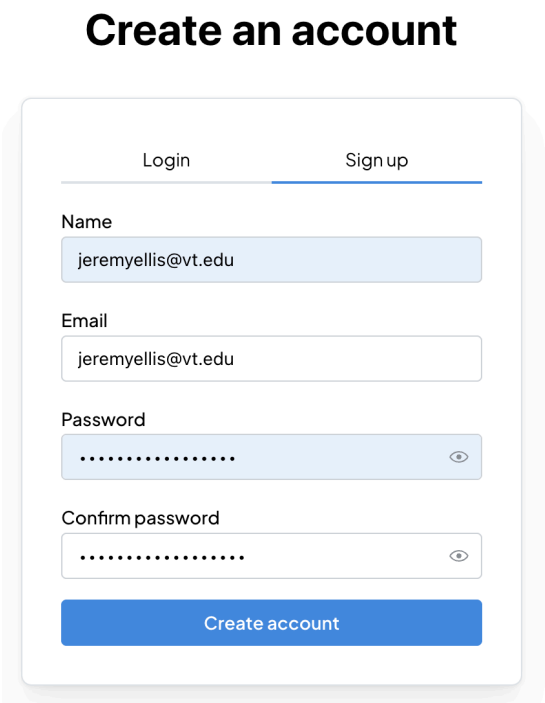


Fig. 4: logging in

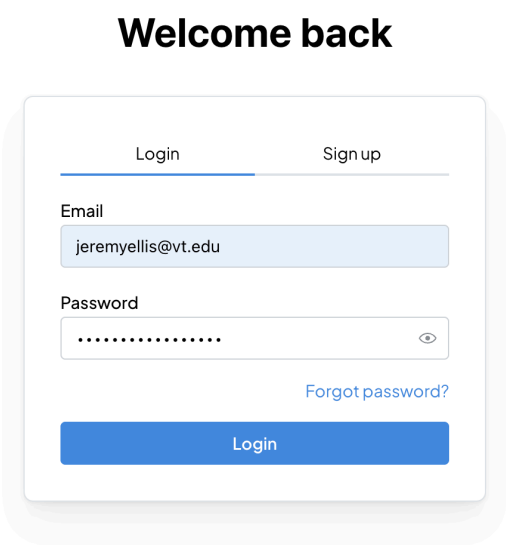


Fig 5. changing password after getting email

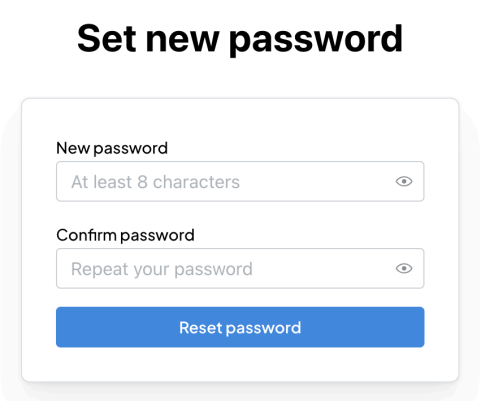
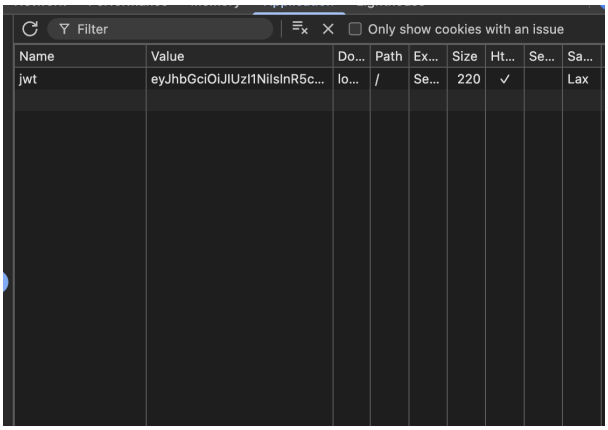


Fig 6. JWT token



2.3 Security Measures

2.3 Security Practices

SQL Injection Prevention

All database queries use parameterized queries via the node-postgres (pg) library. User-supplied values are never concatenated into SQL strings; they are passed as separate arguments using the $1, $2, ... placeholder syntax, and the driver handles escaping entirely.

examples:

```
// From login.ts
const { rows } = await pool.query(
  `SELECT id, email, password_hash, name, is_admin
   FROM accounts WHERE email = $1`,
  [email]
);


// From resetPassword.ts
const { rows } = await pool.query(
  `DELETE FROM password_reset_tokens
   WHERE token = $1 AND expires_at > NOW()
   RETURNING account_id`,
  [token]
);
```

Session Handling

Sessions are managed via JWT stored in an httpOnly cookie. This means:

- JavaScript running in the browser cannot read the token, protecting against XSS-based session theft
- The cookie is flagged secure in production (HTTPS only) and sameSite: none to support cross-origin requests between the frontend and backend deployments
- Logout explicitly clears the cookie server-side rather than relying on the client to discard it
- Tokens expire after 7 days, limiting the window of a stolen token

Avoiding Sensitive Data in URLs

- Login credentials are submitted in the request body (POST), never in the URL
- The JWT session token is stored in a cookie, not a query parameter

- Password reset tokens are included in the URL (/reset-password?token=...) only because this is standard practice for email-based resets. Tokens are single-use, expire in 1 hour, and are deleted from the database the moment they are used

Secure Credential Storage

- Passwords are hashed with bcrypt (cost factor 10) before being stored. The plaintext password is never persisted anywhere
- All application secrets (JWT secret, database credentials, email credentials) are stored in environment variables and loaded via a validated env module. nothing is hardcoded in source code

Fig. 7: env object

```
export const env = {
  PORT: getEnvNumber("PORT"),
  NODE_ENV: getEnv("NODE_ENV"),

  JWT_SECRET: getEnv("JWT_SECRET"),

  ADMIN_PASSWORD: getEnv("ADMIN_PASSWORD"),
  ADMIN_EMAIL: getEnv("ADMIN_EMAIL"),

  DATABASE_PORT: getEnvNumber("DATABASE_PORT"),
  DATABASE_USER: getEnv("DATABASE_USER"),
  DATABASE_PASSWORD: getEnv("DATABASE_PASSWORD"),
  DATABASE_NAME: getEnv("DATABASE_NAME"),
  DATABASE_URL: getEnv("DATABASE_URL"),

  EMAIL: getEnv("EMAIL"),
  EMAIL_PASSWORD: getEnv("EMAIL_PASSWORD"),
} as const;
```

Timing Attack Prevention

During login, if no account is found for the given email, a dummy bcrypt hash is still compared against the submitted password. This ensures the response time is identical whether the email exists or not, preventing an attacker from enumerating valid accounts by measuring response times.

// From login.ts

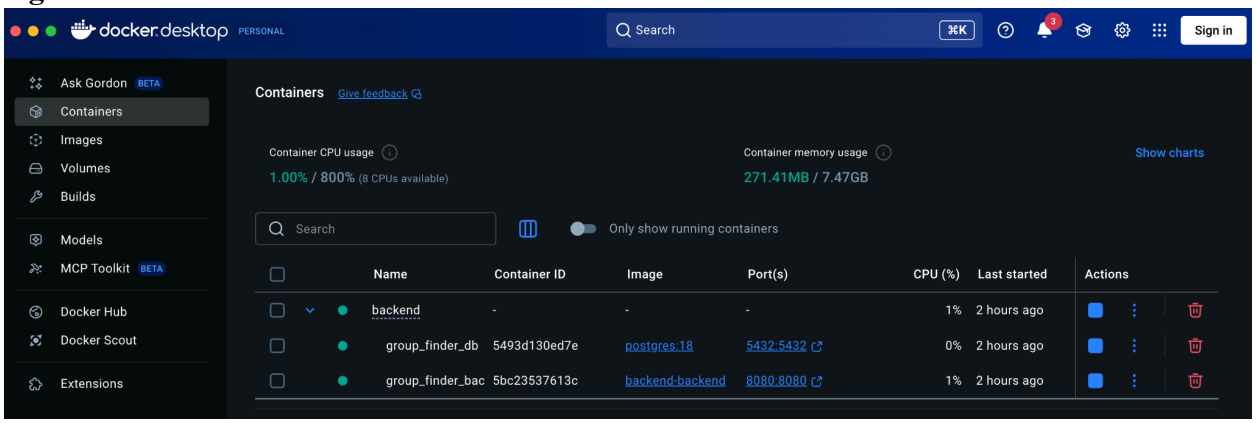
```
  const backUpHash = "$2b$10$stn5DE/DAtvWOGMw4xywfuauxmtsbD7wyXP9/1oEitpFbGinvalid";



  const hash = account?.password_hash ?? backUpHash;

  const valid = await bcrypt.compare(password, hash);
```

3. Database Connection & Setup

3.1 System Connection

The entire system runs in Docker via Docker Compose, which spins up two containers: a PostgreSQL database and a Node.js/Express backend. The schema and seed data are automatically loaded into the database on first run via Docker's init script mechanism.

Fig. 8: Docker container:



The frontend is a React (TypeScript) app built with Vite that communicates with the backend over HTTP REST APIs. The backend connects to PostgreSQL using a node-postgres connection pool, with credentials injected via environment variables.

Fig. 9: frontend communicates to backenv via REST APIs:

```
export const endpoints = {
  // Auth
  signup: (email: string, password: string, name: string) =>
    post<AuthResponse>("/accounts/signup", { email, password, name }),
  login: (email: string, password: string) =>
    post<AuthResponse>("/accounts/login", { email, password }),
  loginMe: () => get<AuthResponse>("/accounts/login/me"),

  // Classes
  getClasses: () => get("/classes"),
  createClass: (name: string, section: string) => post("/classes/add", { name, section }),
  updateClass: (id: number, data: { name?: string; section?: string }) =>
    put(`/classes/update/${id}`, data),
  deleteClass: (id: number) => del(`/classes/delete/${id}`),
```

Stack summary: React + Vite (frontend) → Express/Node (backend) → PostgreSQL (database), containerized with Docker Compose.

3.2 Setup Instructions

Required Tools

- Node.js
- Docker Desktop
- Git

Repo

git clone https://github.com/EllisJeremy/database_group_project

Backend (Node + PostgreSQL)

```
cd backend
npm ci            # install node dependencies
npm run env       # generates a local .env file
npm run docker    # starts the Express server and PostgreSQL in Docker
```

Frontend

```
cd frontend
npm ci
npm run dev
```

3.3 Database Initialization

The database is initialized automatically when Docker starts via two .sql files mounted as PostgreSQL init scripts:

schema.sql — creates all tables and types

seed.sql — inserts sample data (users, classes, groups, posts, skills)

On top of this, seed.ts runs at backend startup to insert the admin account using credentials from environment variables. This is handled in TypeScript rather than SQL so the password is properly bcrypt-hashed before being stored rather than written as plaintext in a .sql file.

3.4 Challenges & Lessons Learned

A big challenge we faced was having the seed data get mutated as we tested the software. The solution was to use Docker to rebuild the environment from scratch every time using SQL and YAML files. Besides this, I have dealt with most of the challenges involved with this project before, so most of it was easy.

4. System Design & Functionality

4.1 Schema Design

Initial Schema (This link is for the diff. To expand the diff so it shows the full file, click the expand button to the right of the file name)

Final Schema

Seed Data

Fig. 10: changes made to schema

Change	Reason
is_admin added to accounts	Admin role was added to support user management; a separate admin table was unnecessary since it's a simple boolean flag
total_slots and filled_slots removed from posts	These were derived/redundant. Slot counts can be computed from account_groups, storing them separately risked data going out of sync
posts.group_id added	Posts were decoupled from groups initially; linking them directly allows a post to advertise a specific group
account_groups.is_pending added	Join requests needed an approval step. Without this flag, there was no way to distinguish pending vs. accepted members
password_reset_tokens added	Password reset functionality required a dedicated table to store short-lived tokens tied to accounts
Primary keys and FK naming standardized	The initial groups table used group_id and account_id inconsistently. The final schema uses id consistently across all tables

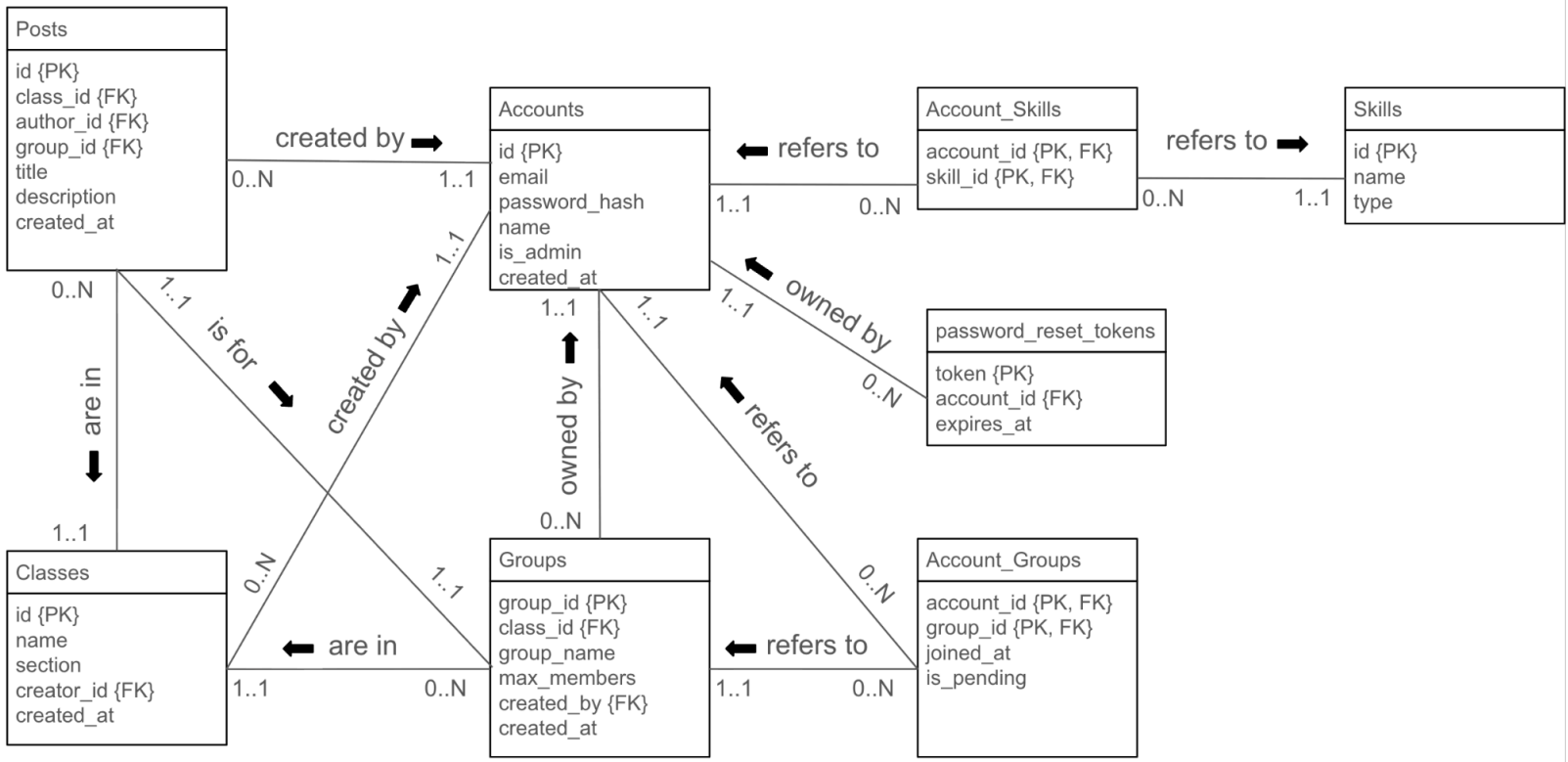
The schema is in Third Normal Form (3NF)
3NF matters here because it eliminates redundant data that can fall out of sync. For example, a filled_slots counter that doesn't match the actual number of rows in account_groups would produce incorrect results without extra logic to keep them in sync.

Important note: In the current version, posts and groups are always created together and function as a single unit. They are implemented as two separate tables to allow for future flexibility, such as posts without groups, groups without public posts, or multiple posts referencing the same group.

4.2 ER Diagram

In the ER diagram, posts and groups are shown as 1..1 to 1..1. This is not enforced at the database level but reflects how the app currently uses these tables. If the intended behavior were to change, the relationship could be updated without modifying the schema or current data in production.

Fig. 11: ER diagram



4.3 Tables & Keys

Fig. 12: accounts

Attribute	Type	Constraints
id	SERIAL	PRIMARY KEY
email	TEXT	UNIQUE, NOT NULL
password_hash	TEXT	NOT NULL
name	TEXT	NOT NULL
is_admin	BOOLEAN	DEFAULT false
created_at	TIMESTAMP	DEFAULT NOW()

Fig. 13: skills

Attribute	Type	Constraints
id	SERIAL	PRIMARY KEY
name	TEXT	UNIQUE, NOT NULL
type	SKILL_TYPE	NOT NULL

Fig. 14: account_skills

Attribute	Type	Constraints
account_id	INT	PRIMARY KEY, FK → accounts(id)
skill_id	INT	PRIMARY KEY, FK → skills(id)

Fig. 15: classes

Attribute	Type	Constraints
id	SERIAL	PRIMARY KEY
name	TEXT	NOT NULL
section	TEXT	NOT NULL
creator_id	INT	NOT NULL, FK → accounts(id)
created_at	TIMESTAMP	DEFAULT NOW()

Fig. 16: groups

Attribute	Type	Constraints
id	SERIAL	PRIMARY KEY
class_id	INT	NOT NULL, FK → classes(id)
group_name	TEXT	NOT NULL
max_members	INT	NOT NULL, CHECK > 0
created_by	INT	NOT NULL, FK → accounts(id)
created_at	TIMESTAMP	DEFAULT NOW()

Fig. 17: posts

Attribute	Type	Constraints
id	SERIAL	PRIMARY KEY
class_id	INT	NOT NULL, FK → classes(id)
author_id	INT	NOT NULL, FK → accounts(id)
group_id	INT	FK → groups(id), ON DELETE SET NULL
title	TEXT	NOT NULL
description	TEXT	NOT NULL
created_at	TIMESTAMP	DEFAULT NOW()

Fig. 18: account_groups

Attribute	Type	Constraints
account_id	INT	PRIMARY KEY, FK → accounts(id)
group_id	INT	PRIMARY KEY, FK → groups(id)
is_pending	BOOLEAN	NOT NULL, DEFAULT true
joined_at	TIMESTAMP	DEFAULT NOW()

Fig. 19: password_reset_tokens

Attribute	Type	Constraints
token	TEXT	PRIMARY KEY
account_id	INT	FK → accounts(id)
expires_at	TIMESTAMPTZ	NOT NULL

4.3 Example Queries

1. Fetch a user's skills

Get a user's skills to be displayed in the skill section on the front end.

```
const { rows } = await pool.query(
  `SELECT s.id, s.name, s.type
  FROM account_skills a_s
  JOIN skills s
  ON a_s.skill_id = s.id
  WHERE a_s.account_id = $1
  `,
  [id],
);
```

2. Fetch users

Get all users and their data to display for the admin page. This route is guarded by require admin middleware, so a normal user can not view all users; only an admin can.

```
const { rows } = await pool.query(`
  SELECT id, email, name, is_admin, created_at
  FROM accounts
  ORDER BY created_at DESC
`);
```

3. Check group capacity before allowing a join request

Before inserting a join request, the app checks whether the group is already full by counting current members against the max.

```
const groupCheck = await pool.query(
  `SELECT g.max_members, g.created_by, COUNT(ag.account_id)::int as member_count
   FROM groups g
   LEFT JOIN account_groups ag ON g.id = ag.group_id
   WHERE g.id = $1
   GROUP BY g.id`,
  [groupId],
);
```

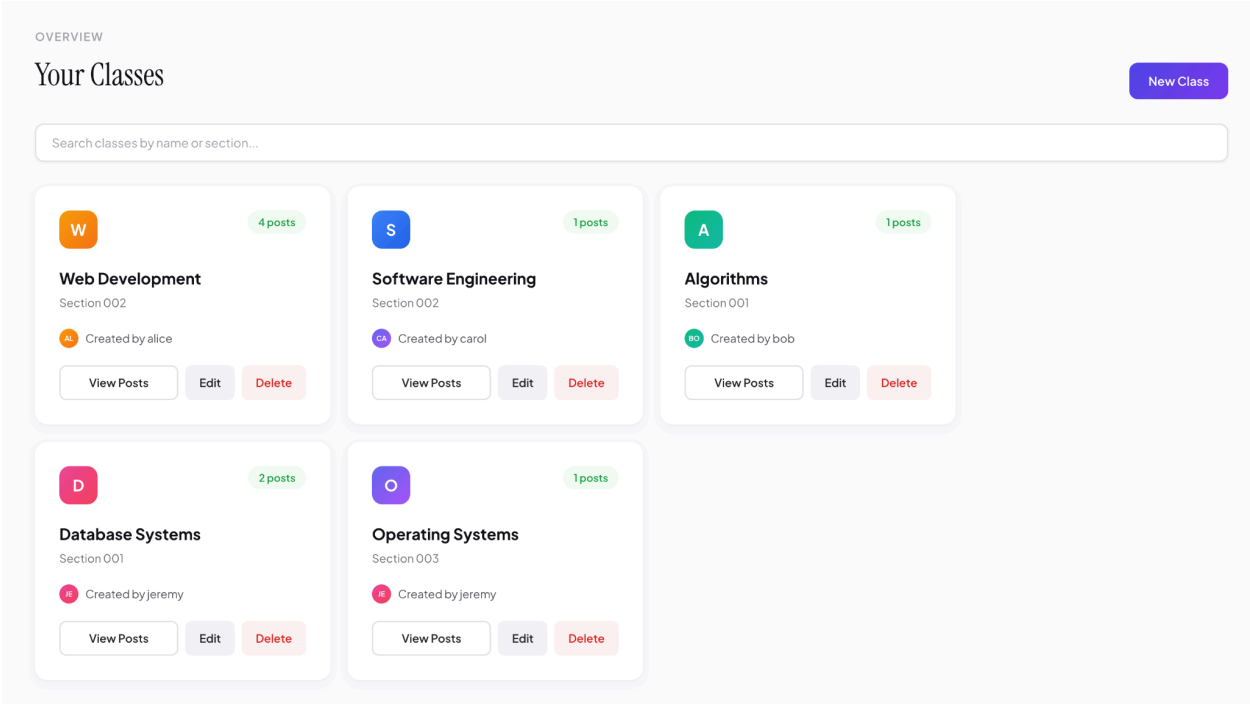
5. Reporting

Report 1: Get all classes

To get the data for the "Your Classes" page, the database queries the accounts, posts, and classes tables to display class names, who created them, and how many posts they have. This is useful to users because it shows them what classes already exist and are joinable.

```
SELECT
  c.*,
  a.name as creator_name,
  COUNT(p.id)::int as post_count
FROM classes c
JOIN accounts a ON c.creator_id = a.id
LEFT JOIN posts p ON p.class_id = c.id
GROUP BY c.id, a.name
ORDER BY c.created_at DESC
```

Fig. 20: Report for all classes

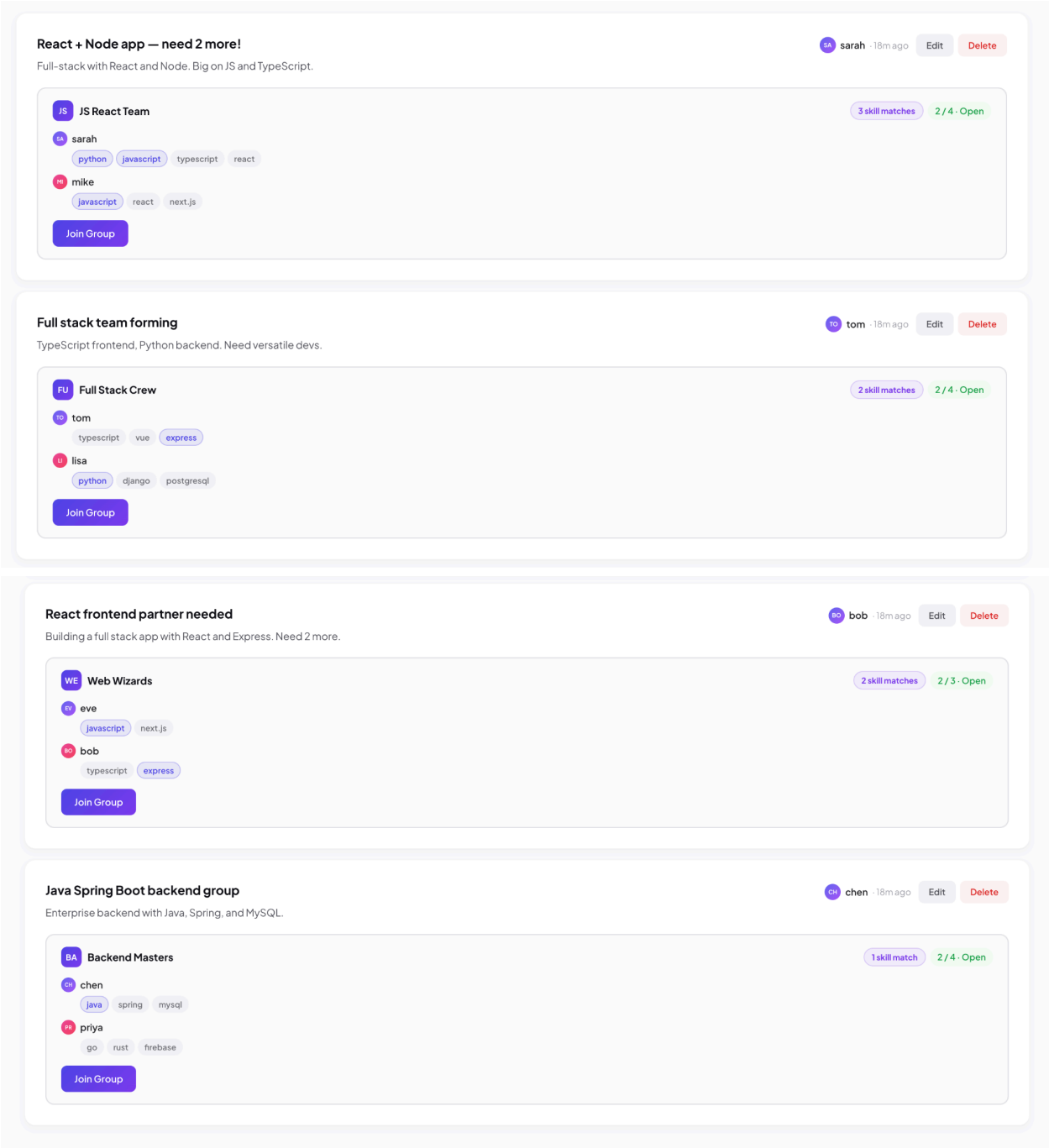


Report 2: Fetch posts with skill-match ranking

When a user loads the posts feed, posts are ranked by how many skills the group's current members share with the logged-in user, so the most relevant groups appear first.

```
const query = `
    WITH user_skills AS (
      SELECT skill_id FROM account_skills WHERE account_id = $1
    )
    SELECT
      p.*,
      a.name as author_name,
      c.name as class_name,
      c.section as class_section,
      g.id as group_id,
      g.group_name,
      g.max_members as group_max_members,
      g.created_by as group_created_by,
      COALESCE((
        SELECT COUNT(*)::int
        FROM account_groups ag2
        JOIN account_skills ags2 ON ag2.account_id = ags2.account_id
        JOIN user_skills us ON ags2.skill_id = us.skill_id
        WHERE ag2.group_id = g.id
        AND ag2.account_id != $1
        AND ag2.is_pending = false
      ), 0) as skill_match_score
    FROM posts p
    JOIN accounts a ON p.author_id = a.id
    JOIN classes c ON p.class_id = c.id
    LEFT JOIN groups g ON p.group_id = g.id
    ${class_id ? "WHERE p.class_id = $2" : ""}
    ORDER BY skill_match_score DESC, p.created_at DESC
  `;
```

Fig. 21: Report for posts



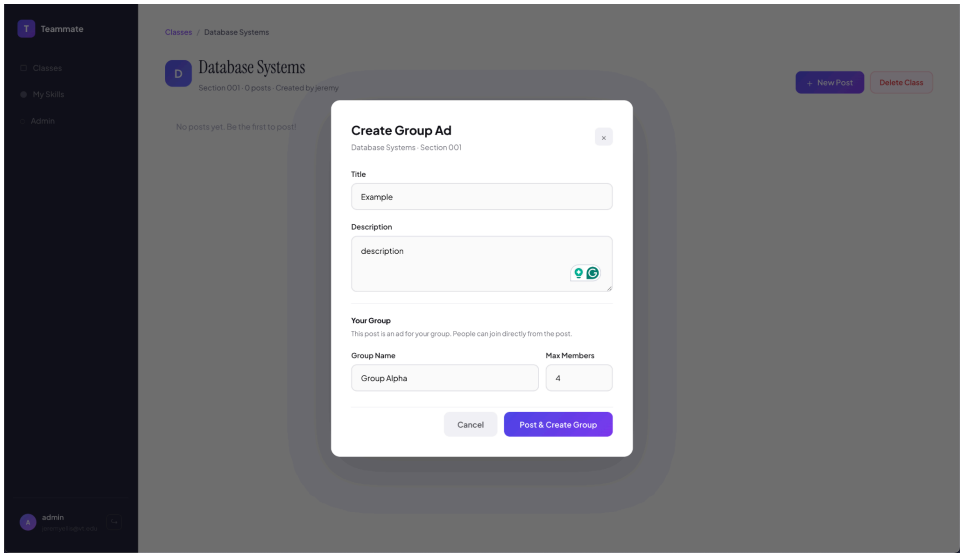
6. GUI / Interface

6.1 CRUD Operations

Here are the 4 CRUD operations using Posts and Groups.

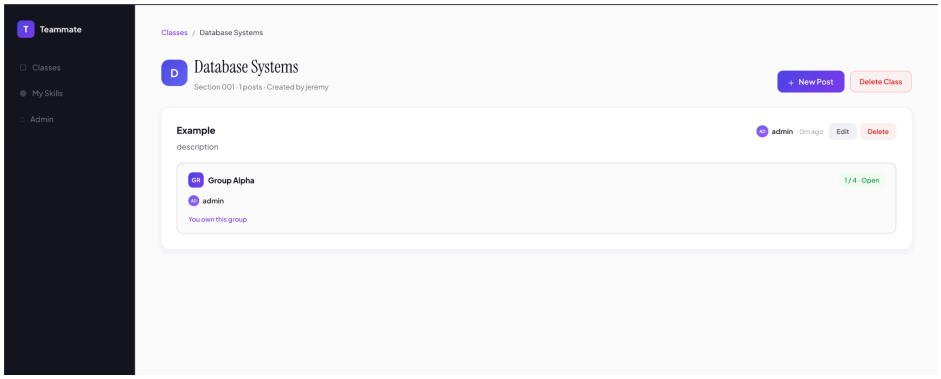
Create:
The user clicks the "New Post" button, fills out the required information, and submits their post and group.

Fig. 22: Create Group Modal



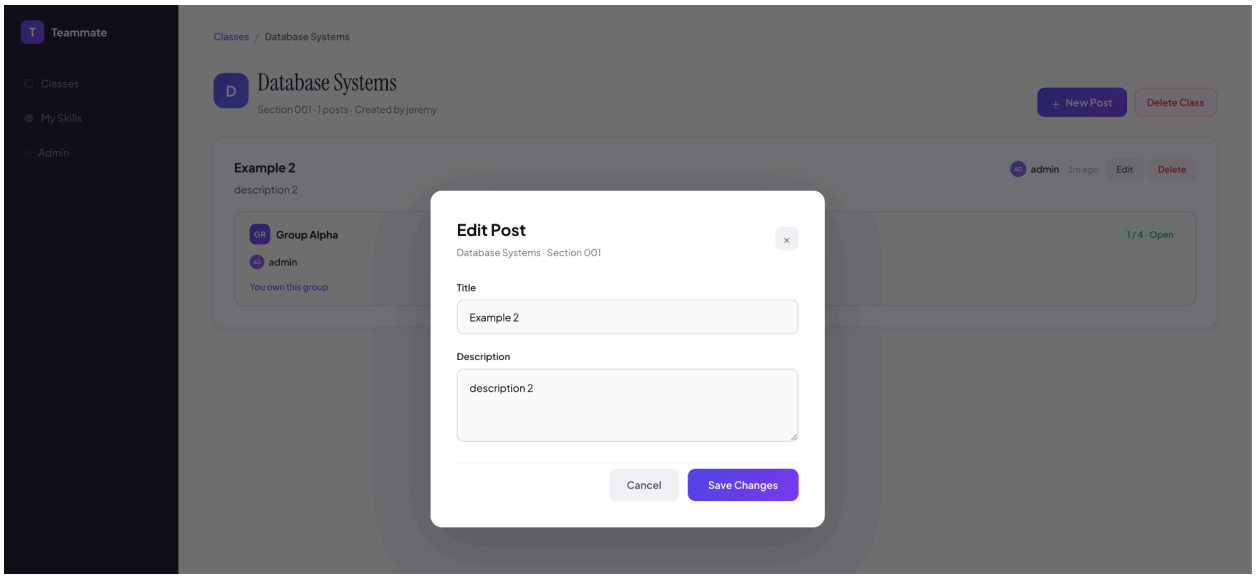
Read 1:
The new entry is read from the database, communicated to the frontend, and displayed in the UI after creation.

Fig. 23: Class and Post UI



Update:
The user clicks the “Edit” button, fills out the information, and saves changes.

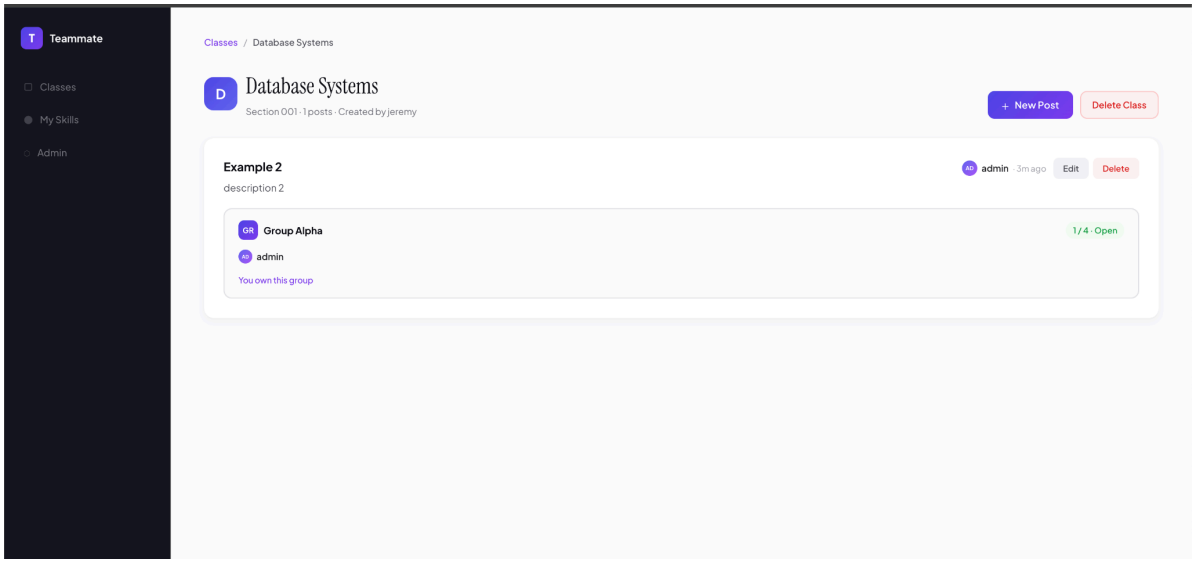
Fig. 24: Edit Post Modal



Read 2:

The UI reflects the changes made.

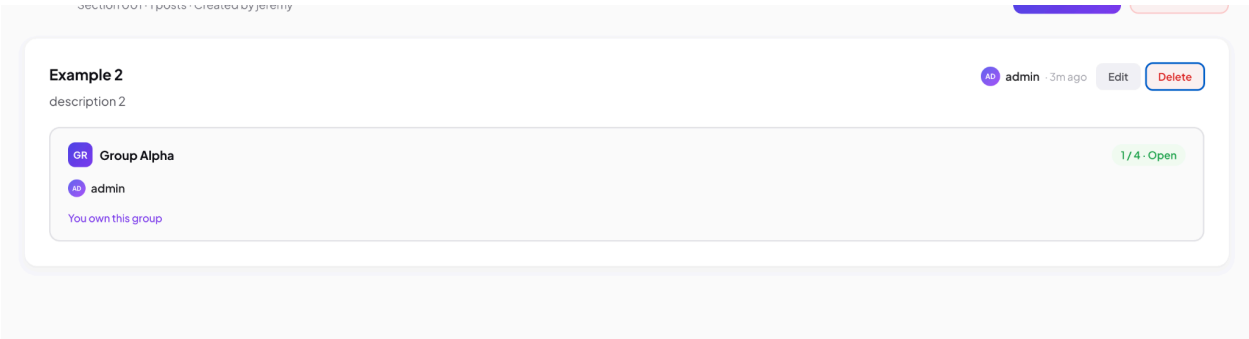
Fig. 25: Class and Post UI After edit



Delete:

The user clicks the “Delete” button.

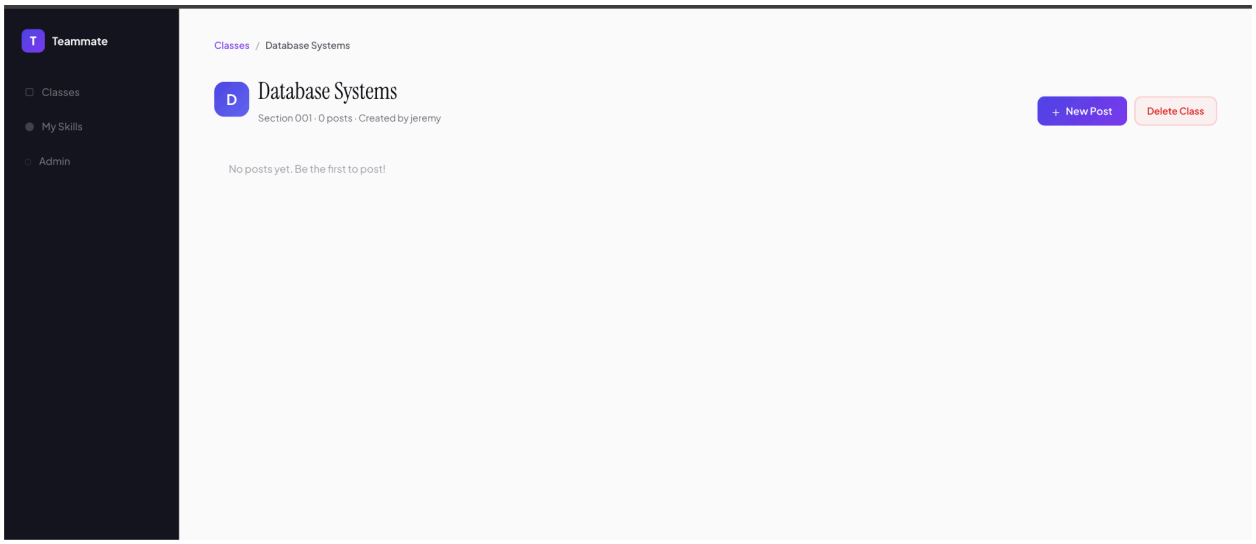
Fig. 26: Delete button



Read 3:

The UI reflects the changes made.

Fig. 27: Class and Post UI After edit



7. Reflection

7.1 Individual Contributions

Jeremy Ellis - I built and architected the full application from scratch, including the database schema design, all backend API routes, and part of the React frontend. On the backend, I implemented user authentication using JWT and bcrypt, role-based access control, password reset via email tokens, and all core features, including classes, posts, groups, and skill management. I containerized the backend stack using Docker Compose, configured the database to auto-initialize with schema and seed data on startup, and set up the development environment with environment variable validation and seeding for the admin account. I also created the entirety of this Report.

Mugilan Murugavel - My role was most of the front-end work. I designed mockups in Paper (Kind of like Figma) for each screen, from creating a post to viewing and commenting on Teammate posts. I then wired this into the code in with the backend.

Isaac Han - My role was to be a filler, helping out with tedious work as asked. I designed diagrams and wrote documentation.

Nitant Rai: I contributed to the documentation and ER diagram. I worked on the report by drafting and organizing sections and overall documentation.

7.2 Team Reflection

One feature the team is particularly proud of is the skill-based post ranking system. Rather than displaying recruitment posts in a flat chronological feed, the platform automatically surfaces posts from students whose skills best match what a team needs. This required combining profile data, skill associations, and post metadata into a single ranked query, and getting that to feel seamless in the UI was a meaningful technical challenge. It also directly addresses the core problem the app was built to solve, so seeing it work end-to-end was satisfying.

One alternative approach we considered was separating teacher and student into distinct roles rather than using a single account type with an admin flag. In hindsight, keeping it as one flexible role simplified the schema and avoided over-engineering a distinction that did not add much value for the scope of this project. Any user can create a class or post, which mirrors how real academic collaboration actually works.

The biggest challenge throughout the project was keeping seed data from getting corrupted during testing. Every time we ran manual tests, the database state would drift and become inconsistent, which made it hard to reproduce bugs or demo features reliably. Switching to a Docker-based setup that rebuilds the database from schema.sql and seed.sql on every fresh start

solved this completely and ended up being one of the better architectural decisions we made. If we were to extend the project, we would invest earlier in a proper testing environment so development and demo data stay cleanly separated

8. Appendix

Repo: https://github.com/EllisJeremy/Database_Group_Project/